Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Lab Report

on

"Operating Systems Lab-III"

[Code No.: COMP 307]

Submitted by

Prajwal Ghimire
Roll No.: 22

Submitted to

Mr. Rabina Shrestha
Department of Computer Science and Engineering

January 11, 2026

Lab Report 3: Simulation of Process Scheduling Algorithm

1 Scheduling Concepts

Objective: To understand and simulate CPU scheduling algorithms.

1.1 Preemptive vs. Non-preemptive Scheduling

- **Non-preemptive Scheduling:** In this mode, once a process has been allocated the CPU, it keeps the CPU until it releases it either by terminating or by switching to the waiting state. The operating system cannot forcibly take the CPU away. Examples include First-Come-First-Serve (FCFS) and standard Shortest Job First (SJF).
- **Preemptive Scheduling:** In this mode, the operating system can interrupt a currently running process and move it to the ready queue to allocate the CPU to another process. This is often done based on priority or time quantum. Examples include Round Robin (RR) and Shortest Remaining Time First (SRTF).

2 Shortest Job First (SJF)

Algorithm Overview: SJF is a non-preemptive scheduling algorithm that selects the waiting process with the smallest execution time (burst time) to execute next. This approach is optimal for minimizing the average waiting time for a given set of processes. However, it can lead to starvation for long processes if shorter processes keep arriving.

2.1 Source Code

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;

    int burst_time[20], arrival_time[20], waiting_time[20], turnaround_time[20], pid[20];
    int is_completed[20] = {0};

    // Gantt chart tracking
    int gantt_pid[20], gantt_start[20], gantt_end[20];
    int gantt_index = 0;

    float avg_waiting_time = 0;
    float avg_turn_around_time = 0;

    printf("Enter the no of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter arrival time of P%d: ", pid[i]);
        scanf("%d", &arrival_time[i]);
        printf("Enter burst time of P%d: ", pid[i]);
        scanf("%d", &burst_time[i]);
    }

    while (completed != n) {
        int index = -1;
        int min_burst_time = 99999;

        for (i = 0; i < n; i++) {
            if (arrival_time[i] <= time && !is_completed[i]) {
                if (burst_time[i] < min_burst_time) {
                    min_burst_time = burst_time[i];
                    index = i;
                }
            }
        }

        if (index == -1) {
            time++;
        } else {
            gantt_pid[gantt_index] = pid[index];
            gantt_start[gantt_index] = time;

            waiting_time[index] = time - arrival_time[index];
            time += burst_time[index];
            turnaround_time[index] = waiting_time[index] + burst_time[index];

            gantt_end[gantt_index] = time;
            gantt_index++;

            is_completed[index] = 1;
            completed++;
        }
    }
```

```c
57
58      printf("\nPID\tAT\tBT\tWT\tTAT\n");
59      for (i = 0; i < n; i++) {
60          avg_waiting_time += waiting_time[i];
61          avg_turn_around_time += turnaround_time[i];
62
63          printf("P%d\t%d\t%d\t%d\t%d\n",
64                 pid[i], arrival_time[i], burst_time[i],
65                 waiting_time[i], turnaround_time[i]);
66      }
67
68      printf("\nAverage Waiting Time = %.2f", avg_waiting_time / n);
69      printf("\nAverage Turnaround Time = %.2f\n", avg_turn_around_time / n);
70
71      // Gantt Chart
72      printf("\nGantt Chart:\n");
73      printf("------------------------------------------------\n");
74
75      for (i = 0; i < gantt_index; i++) {
76          printf("| P%d ", gantt_pid[i]);
77      }
78      printf("|\n");
79
80      printf("------------------------------------------------\n");
81
82      printf("%d", gantt_start[0]);
83      for (i = 0; i < gantt_index; i++) {
84          printf("    %d", gantt_end[i]);
85      }
86      printf("\n");
87
88      return 0;
89  }
90
```

2.2 Output

```
~/os/lab/lab_3 on  master ?1
> ./sjf
Enter the no of processes: 3
Enter arrival time of P1: 0
Enter burst time of P1: 3
Enter arrival time of P2: 2
Enter burst time of P2: 5
Enter arrival time of P3: 5
Enter burst time of P3: 6

PID     AT      BT      WT      TAT
P1      0       3       0       3
P2      2       5       1       6
P3      5       6       3       9

Average Waiting Time = 1.33
Average Turnaround Time = 6.00

Gantt Chart:
------------------------------
| P1 | P2 | P3 |
------------------------------
0    3    8    14
```

Figure 1: Output of SJF Simulation

3 Shortest Remaining Time First (SRTF)

Algorithm Overview: SRTF is the **preemptive** version of the SJF algorithm. In this method, the process with the smallest amount of time remaining until completion is selected to execute. If a new process arrives with a CPU burst time less than the remaining time of the currently executing process, the current process is preempted.

3.1 Source Code

```c
#include <stdio.h>

    int main() {
        int n, i, time = 0, completed = 0;
        int burst_time[20], remaining_time[20], arrival_time[20];
        int waiting_time[20], turnaround_time[20], pid[20];
        int is_completed[20] = {0};

        // Gantt chart
        int gantt_pid[100], gantt_time[100];
        int g = 0;

        float avg_wt = 0, avg_tat = 0;

        printf("Enter number of processes: ");
        scanf("%d", &n);

        for (i = 0; i < n; i++) {
            pid[i] = i + 1;
            printf("Enter arrival time of P%d: ", pid[i]);
            scanf("%d", &arrival_time[i]);
            printf("Enter burst time of P%d: ", pid[i]);
            scanf("%d", &burst_time[i]);
            remaining_time[i] = burst_time[i];
        }

        while (completed != n) {
            int index = -1;
            int min_rt = 99999;

            for (i = 0; i < n; i++) {
                if (arrival_time[i] <= time && remaining_time[i] > 0) {
                    if (remaining_time[i] < min_rt) {
                        min_rt = remaining_time[i];
                        index = i;
                    }
                }
            }

            if (index == -1) {
                time++;
            } else {
                gantt_pid[g] = pid[index];
                gantt_time[g++] = time;

                remaining_time[index]--;
                time++;

                if (remaining_time[index] == 0) {
                    completed++;
                    turnaround_time[index] = time - arrival_time[index];
                    waiting_time[index] = turnaround_time[index] - burst_time[index];
                }
            }
        }
```

```c
        printf("\nPID\tAT\tBT\tWT\tTAT\n");
        for (i = 0; i < n; i++) {
            avg_wt += waiting_time[i];
            avg_tat += turnaround_time[i];
            printf("P%d\t%d\t%d\t%d\t%d\n",
                   pid[i], arrival_time[i], burst_time[i],
                   waiting_time[i], turnaround_time[i]);
        }

        printf("\nAverage Waiting Time = %.2f", avg_wt / n);
        printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

        // Gantt Chart
        printf("\nGantt Chart:\n");
        for (i = 0; i < g; i++) {
            printf("| P%d ", gantt_pid[i]);
        }
        printf("|\n");

        return 0;
    }
```

3.2 Output

```
~/os/lab/lab_3 on master ?1
❯ ./srtf
Enter number of processes: 3
Enter arrival time of P1: 0
Enter burst time of P1: 5
Enter arrival time of P2: 2
Enter burst time of P2: 2
Enter arrival time of P3: 4
Enter burst time of P3: 1

PID     AT      BT      WT      TAT
P1      0       5       3       8
P2      2       2       0       2
P3      4       1       0       1

Average Waiting Time = 1.00
Average Turnaround Time = 3.67

Gantt Chart:
| P1 | P1 | P2 | P2 | P3 | P1 | P1 | P1 |
```

Figure 2: Output of SRTF Simulation

4 Round Robin (RR)

Algorithm Overview: Round Robin is a preemptive scheduling algorithm designed specifically for time-sharing systems. Each process is assigned a small unit of time called a **Time Quantum** (or Time Slice). The CPU scheduler cycles through the ready queue, allocating the CPU to each process for a time interval of up to one time quantum. If the process does not finish within the quantum, it is preempted and placed at the back of the ready queue.

4.1 Source Code

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, tq;
    int burst_time[20], remaining_time[20], arrival_time[20];
    int waiting_time[20] = {0}, turnaround_time[20], pid[20];
    int completed = 0;

    // Gantt chart
    int gantt_pid[100], gantt_time[100];
    int g = 0;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter arrival time of P%d: ", pid[i]);
        scanf("%d", &arrival_time[i]);
        printf("Enter burst time of P%d: ", pid[i]);
        scanf("%d", &burst_time[i]);
        remaining_time[i] = burst_time[i];
    }

    while (completed != n) {
        int done = 1;

        for (i = 0; i < n; i++) {
            if (arrival_time[i] <= time && remaining_time[i] > 0) {
                done = 0;

                gantt_pid[g] = pid[i];
                gantt_time[g++] = time;

                if (remaining_time[i] > tq) {
                    time += tq;
                    remaining_time[i] -= tq;
                } else {
                    time += remaining_time[i];
                    waiting_time[i] = time - arrival_time[i] - burst_time[i];
                    remaining_time[i] = 0;
                    completed++;
                }
            }
        }

        if (done)
            time++;
    }
```

```c
    for (i = 0; i < n; i++) {
        turnaround_time[i] = waiting_time[i] + burst_time[i];
        avg_wt += waiting_time[i];
        avg_tat += turnaround_time[i];
    }

    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n",
               pid[i], arrival_time[i], burst_time[i],
               waiting_time[i], turnaround_time[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

    // Gantt Chart
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        printf("| P%d ", gantt_pid[i]);
    }
    printf("|\n");

    return 0;
}
```

4.2 Output

```
@ ~/os/lab/lab_3 on master ?1
❯ ./rr
Enter number of processes: 4
Enter Time Quantum: 2
Enter arrival time of P1: 0
Enter burst time of P1: 5
Enter arrival time of P2: 3
Enter burst time of P2: 4
Enter arrival time of P3: 3
Enter burst time of P3: 5
Enter arrival time of P4: 5
Enter burst time of P4: 6

PID     AT      BT      WT      TAT
P1      0       5       6       11
P2      3       4       6       10
P3      3       5       10      15
P4      5       6       9       15

Average Waiting Time = 7.75
Average Turnaround Time = 12.75

Gantt Chart:
| P1 | P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P3 | P4 |
```

Figure 3: Output of Round Robin Simulation